# Reverse Engineering of Object Oriented Code

## Monographs in Computer Science

Abadi and Cardelli, **A Theory of Objects**

Benosman and Kang [editors], **Panoramic Vision: Sensors, Theory and Applications**

Broy and Stølen, **Specification and Development of Interactive Systems: FOCUS on Streams, Interfaces, and Refinement**

Brzozowski and Seger, **Asynchronous Circuits**

Cantone, Omodeo, and Policriti, **Set Theory for Computing: From Decision Procedures to Declarative Programming with Sets**

Castillo, Gutiérrez, and Hadi, Expert **Systems and Probabilistic Network Models**

Downey and Fellows, **Parameterized Complexity**

Feijen and van Gasteren, **On a Method of Multiprogramming**

Herbert and Spärck Jones [editors], **Computer Systems: Theory, Technology, and Applications**

Leiss, **Language Equations**

McIver and Morgan [editors], **Programming Methodology**

McIver and Morgan, **Abstraction, Refinement and Proof for Probabilistic Systems**

Misra, **A Discipline of Multiprogramming: Program Theory for Distributed Applications**

Nielson [editor], **ML with Concurrency**

Paton [editor], **Active Rules in Database Systems**

Selig, **Geometric Fundamentals of Robotics, Second Edition**

Tonella and Potrich, **Reverse Engineering of Object Oriented Code**

Paolo Tonella
Alessandra Potrich

# Reverse Engineering of Object Oriented Code

Springer

Paolo Tonella and Alessandra Potrich
ITC-irst
Via Sommarive
Povo, Trent 38050
ITALY



*Cover illustration*: Verona-climbing the tower. Photo courtesy Philip Greenspun, http://philip.greenspun.com.

ISBN 0-387-40295-0 e-ISBN 0-387- 23803-4 Printed on acid-free paper.



Printed in the United States of America. (BS/DH)

9 8 7 6 5 4 3 2 1 SPIN 10935804

springeronline.com

*To Silvia and Chiara*
*– Paolo*

*To Bruno*
*– Alessandra*

# Contents

# Foreword

There has been an ongoing debate on how best to document a software system ever since the first software system was built. Some would have us writing natural language descriptions, some would have us prepare formal specifications, others would have us producing design documents and others would want us to describe the software thru test cases. There are even those who would have us do all four, writing natural language documents, writing formal specifications, producing standard design documents and producing interpretable test cases all in addition to developing and maintaining the code. The problem with this is that whatever is produced in the way of documentation becomes in a short time useless, unless it is maintained parallel to the code. Maintaining alternate views of complex systems becomes very expensive and highly error prone. The views tend to drift apart and become inconsistent.

The authors of this book provide a simple solution to this perennial problem. Only the source code is maintained and evolved. All of the other information required on the system is taken from the source code. This entails generating a complete set of UML diagrams from the source. In this way, the design documentation will always reflect the real system as it is and not the way the system should be from the viewpoint of the documentor. There can be no inconsistency between design and implementation. The method used is that of reverse engineering, the target of the method is object oriented code in C++, C#, or Java. From the code class diagrams, object diagrams, interaction diagrams and state diagrams are generated in accordance with the latest UML standard. Since the method is automated, there are no additional costs. Design documentation is provided at the click of a button.

This approach, the result of many years of research and development, will have a profound impact upon the way IT-systems are documented. Besides the source code itself, only one other view of the system needs to be developed and maintained, that is the user view in the form of a domain specific language. Each application domain will have to come up with it's own language to describe applications from the view point of the user. These languages may range from natural languages to set theory to formal mathematical notations.

What these languages will not describe is how the system is or should be constructed. This is the purpose of UML as a modeling language. The techniques described in this book demonstrate that this design documentation can and should be extracted from the code, since this is the cheapest and most reliable means of achieving this end. There may be some UML documents produced on the way to the code, but since complex IT systems are almost always developed by trial and error, these documents will only have a transitive nature. The moment the code exists they are both obsolete and superfluous. From then on, the same documents can be produced cheaper and better from the code itself. This approach coincides with and supports the practice of extreme programming.

Of course there are several drawbacks, as some types of information are not captured in the code and, therefore, reverse engineering cannot capture them. An example is that there still needs to be a test oracle – something to test against. This something is the domain specific specification from which the application-oriented test cases are derived. The technical test cases can be derived from the generated UML diagrams. In this way, the system as implemented will be verified against the system as specified. Without the UML diagrams, extracted from the code, there would be no adequate basis of comparison.

For these and other reasons, this book is highly recommendable to all who are developing and maintaining Object-Oriented software systems. They should be aware of the possibilities and limitations of automated post documentation. It will become increasing significant in the years to come, as the current generation of OO-systems become the legacy systems of the future. The implementation knowledge they encompass will most likely be only in the source and there will be no other means of regaining it other than through reverse engineering.

Trento, Italy, July 2004 *Harry Sneed*
Benevento, Italy, July 2004 *Aniello Cimitile*

# Preface

Diagrams representing the organization and behavior of an Object Oriented software system can help developers comprehend it and evaluate the impact of a modification. However, such diagrams are often unavailable or inconsistent with the code. Their extraction from the code is thus an appealing option. This book represents the state of the art of the research in Object Oriented code analysis for reverse engineering. It describes the algorithms involved in the recovery of several alternative views from the code and some of the techniques that can be adopted for their visualization.

During software evolution, availability of high level descriptions is extremely desirable, in support to program understanding and to change-impact analysis. In fact, location of a change to be implemented can be guided by high level views. The dependences among entities in such views indicate the proportion of the ripple effects.

However, it is often the case that diagrams available during software evolution are not consistent with the code, or – even more frequently – that no diagram has altogether been produced. In such contexts, it is crucial to be able to reverse engineer design diagrams directly from the code. Reverse engineered diagrams are a faithful representation of the actual code organization and of the actual interactions among objects. Programmers do not face any misalignment or gap when moving from such diagrams to the code.

The material presented in this book is based on the techniques developed during a collaboration we had with CERN (Conseil Européen pour la Recherche Nucléaire). At CERN, work for the next generation of experiments to be run on the Large Hadron Collider has started in large advance, since these experiments represent a major challenge, for the size of the devices, teams, and software involved. We collaborated with CERN in the introduction of tools for software quality assurance, among which a reverse engineering tool.

The algorithms described in this book deal with the reverse engineering of the following diagrams:

**Class diagram:** Extraction of inter-class relationships in presence of weakly typed containers and interfaces, which prevent an exact knowledge of the actual type of referenced objects.

**Object and interaction diagrams:** Recovery of the associations among the objects that instantiate the classes in a system and of the messages exchanged among them.

**State diagram:** Modeling of the behavior of each class in terms of states and state transitions.

**Package diagram:** Identification of packages and of the dependences among packages.

All the algorithms share a common code analysis framework. The basic principle underlying such a framework is that information is derived *statically* (no code execution) by performing a propagation of proper data in a graph representation of the object flows occurring in a program. The data structure that has been defined for such a purpose is called the Object Flow Graph (OFG). It allows tracking the lifetime of the objects from their creation along their assignment to program variables.

UML, the Unified Modeling Language, has been chosen as the graphical language to present the outcome of reverse engineering. This choice was motivated by the fact that UML has become the standard for the representation of design diagrams in Object Oriented development. However, the choice of UML is by no means restrictive, in that the same information recovered from the code can be provided to the users in different graphical or non graphical formats.

A well known concern of most reverse engineering methods is how to filter the results, when their size and complexity are excessively high. Since the recovered diagrams are intended to be inspected by a human, the presentation modes should take into account the cognitive limitations of humans explicitly. Techniques such as focusing, hierarchical structuring and element explosion/implosion will be introduced specifically for some diagram types.

The research community working in the field of reverse engineering has produced an impressive amount of knowledge related to techniques and tools that can be used during software evolution in support of program understanding. It is the authors' opinion that an important step forward would be to publish the achievements obtained so far in comprehensive books dealing with specific subtopics.

This book on reverse engineering from Object Oriented code goes exactly in this direction. The authors have produced several research papers in this field over time and have been active in the research community. The techniques and the algorithms described in the book represent the current state of the art.

Trento, Italy — *Paolo Tonella*
July 2004 — *Alessandra Potrich*

# 1

# Introduction

Reverse engineering aims at supporting program comprehension, by exploiting the source code as the major source of information about the organization and behavior of a program, and by extracting a set of potentially useful views provided to programmers in the form of diagrams. Alternative perspectives can be adopted when the source code is analyzed and different higher level views are extracted from it. The focus may either be on the structure, on the behavior, on the internal states, or on the physical organization of the files. A single diagram recovered from the code through reverse engineering is insufficient. Rather, a set of complementary views need to be obtained, addressing different program understanding needs.

In this chapter, the role of reverse engineering within the life cycle of a software system is described. The activities of program understanding and impact analysis are central during the evolution of an existing system. Both activities can benefit from sources of knowledge about the program such as reverse engineered diagrams.

The reverse engineering techniques presented in the following chapters are described with reference to an example program used throughout the book. In this chapter, this example program is introduced and commented. Then, some of the diagrams that are the object of the following chapters are provided for the example program, showing their usefulness from the programmer's point of view. The remaining parts of the book contain the algorithmic details on how to recover them from the source code.

## 1.1 Reverse Engineering

In the life cycle of a software system, the maintenance phase is the largest and the most expensive. Starting after the delivery of the first version of the software [35], maintenance lasts much longer than the initial development phase. During this time, the software will be changed and enhanced over and over. So it is more appropriate to speak of *software evolution* with reference

to the whole life cycle, in which the initial development is only a special case where the existing system is empty.

Software evolution is characterized by the existence of the source code of the system. Thus, the typical activity in software evolution is the implementation of a program change, in response to a *change request*. Changes may be aimed at correcting the software (*corrective* maintenance), at adding a functionality (*perfective* maintenance), at adapting the software to a changed environment (*adaptive* maintenance), or at restructuring it to make future maintenance easier (*preventive* maintenance) [35].

During software evolution, the most reliable and accurate description of the behavior of a software system is its source code. In fact, design diagrams are often outdated or missing at all. Such a valuable information repository may not directly answer all questions about the system. Reverse engineering techniques provide a way to extract higher level views of the system, which summarize some relevant aspects of the computation performed by the program statements. Reverse engineered diagrams support program comprehension, as well as restructuring and traceability.

When an existing code base is worked on, the micro-process of *program change* can be decomposed into localizing the change, assessing the impact, and implementing the change. All such activities depend on the knowledge available about the program to be modified. In this respect, reverse engineering techniques are a useful support. Reverse engineering tools provide useful high level information about the system being maintained, thus helping programmers locate the component to be modified. Moreover, the relationships (dependencies, associations, etc.) that connect the entities in reverse engineered diagrams provide indications about the impact of a change. By tracing such relationships the set of entities possibly affected by a change are obtained.

Object Oriented programming poses special problems to software engineers during the maintenance phase. Correspondingly, reverse engineering techniques have to be customized to address them. For example, the behavior of an Object Oriented program emerges from the interactions occurring among the objects allocated in the program. The related instructions may be spread across several classes, which individually perform a very limited portion of the work locally and delegate the rest of it to others. Reverse engineered diagrams capture such collaborations among classes/objects, summarizing them in a single, compact view. However, recovering accurate information about such collaborations represents a special challenge, requiring major improvements to the available reverse engineering methods [48, 100].

When a software system is analyzed to extract information about it, the fundamental choice is between static and dynamic analysis. *Dynamic* analysis requires a tracer tool to save information about the objects manipulated and the methods dispatched during program execution. The diagrams that can be reverse engineered in this way are partial. They hold valid for a single, given execution of the program, with given input values, and they cannot be easily generalized to the behavior of the program for any execution with any

input. Moreover, dynamic analysis is possible only for complete, executable systems, while in Object Oriented programming it is typical to produce incomplete sets of classes that are reused in different contexts. On the contrary, a *static* analysis produces results that are valid for all executions and for all inputs. On the other side, static analyses may be over-conservative. In fact, it is undecidable to determine if a statically possible path is *feasible*, i.e., if there exists an input value allowing its traversal. Static analysis may conservatively assume that some paths are executable, while they are actually not so. Consequently, it may produce results for which no input value exists. In the following chapters, the advantages and disadvantages of the two approaches will be discussed for each specific diagram, illustrating them on an executable example.

UML (Unified Modeling Language) [7, 69] has become the standard graphical language used to represent Object Oriented systems in diagrammatic form. Its specifications have been recently standardized by the Object Management Group (OMG) [1]. UML has been adopted by several software companies, and its theoretical aspects are the subject of several research studies. For these reasons, UML was chosen as the graphical representation that is produced as the output of the reverse engineering techniques described in this book. However, the choice of UML is by no means limiting: while the information reverse engineered from the code can be represented in different graphical (or non graphical) forms, the basic analysis methods exploited to produce it can be reused unchanged in alternative settings, with UML replaced by some other description language.

An important issue reverse engineering techniques must take into account is usability. Since the recovered views are for humans and not for computers, they must be compatible with the cognitive abilities of human beings. This means that diagrams convey useful information only if their size is kept small (while 10 entities may be fine, 100 starts being too much and 1000 makes a diagram unreadable). Several approaches can be adopted to support visualization and navigation modes making reverse engineered information usable. They range from the possibility to focus on a portion of the system, to the expand/collapse or zoom in/out operations, or to the availability of an overall navigation map complemented by a detailed view. In the following chapters, ad hoc methods will be described with reference to the specific diagrams being produced.

## 1.2 The *eLib* Program

The *eLib* program is a small Java program that supports the main functions operated in a library. Its code is provided in Appendix A. It will be used in the remaining of this book as the example.

In *eLib*, libraries are supposed to hold an archive of documents of different categories, properly classified. Each document can be uniquely identified by

the librarian. Library users can request some of these documents for loan, subjected to proper access rules. In order to borrow a document, users must be identified by the librarian. For example, this could be achieved by distributing library cards to registered users.

As regards the management of the documents in the *eLib* system, the librarian can insert new documents in the archive and remove documents no longer available in the library. Upon request, the librarian may need to search the archive for documents according to some search criterion, such as title, authors, ISBN code, etc. The documents held by a library are of several different kinds, including books, journals, and technical reports. Each of them has specific properties and specific access restrictions.

As far as user management is concerned, a set of personal data (name, address, phone number, etc.) are maintained in the archive. A special category of users consists of internal users, who have special permission to access documents not allowed for loan to normal users.

The main functionality of the *eLib* system is loan management. Users can borrow documents up to a maximum number. While books are available for loan to any user, journals can be borrowed only by internal users, and technical reports can be consulted but not borrowed.

Although this is a small application, by going through the source code of the *eLib* program (see Appendix A) it is not so easy to understand how the classes are organized, how they interact with each other to fulfill the main functions, how responsibilities are distributed among the classes, what is computed locally and what is delegated. For example, a programmer aiming at understanding this application may have the following questions:

- What is the overall system organization?
- What objects are updated when a document is borrowed?
- What classes are responsible to check if a given document can be borrowed by a given user?
- How is the maximum number of loans handled?
- What happens to the state of the library when a document is returned?

Let us assume the following change request (perfective maintenance):

> *When a document is not available for loan, a user can reserve it, if it has not been previously reserved by another user. When a document is returned to the library, the user who reserved it is contacted, if any is associated with the document. The user can either borrow the document that has become available or cancel the reservation. In both cases, after this operation the reservation of the document is deleted.*

the programmer who is responsible for its implementation may have the following questions about the system:

- Does the overall system organization need any change?
- What classes need to collaborate to realize the reservation functionality?

- Is there any possible side effect on the existing functionalities?
- What changes should be made in the procedure for returning documents to the library?
- How is the new state of a document described?
- Is there any interaction between the new rules for document borrowing and the existing ones?

In the following sections, we will see how UML diagrams reverse engineered from the code can help answer the program understanding and impact analysis questions listed above.

## 1.3 Class Diagram

The class diagram reverse engineered from the code helps understand the overall system's organization and the kind of interclass connections that exist in the program.

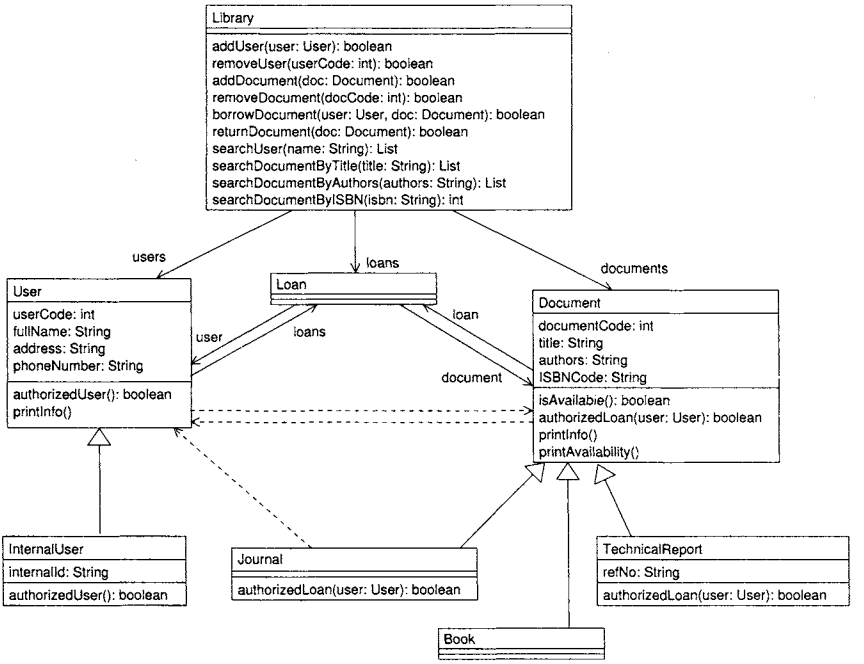


**Fig. 1.1.** Class diagram for the *eLib* program.

Fig. 1.1 shows the class diagram of the *eLib* program, including all interclass dependencies. The UML graphical language has been adopted, so that

dashed lines indicate a dependency, solid lines an association and empty arrows inheritance. The exact meaning of the notation will be clarified in the following chapters. An intuitive idea is sufficient for the purposes of this section. Only some attributes and methods inside the compartments of each class have been selected for display.

The overall architecture of the system is clear from Fig. 1.1. The class `Library` provides the main functionalities of the *eLib* program. For example, library users are managed through the methods `addUser` and `removeUser`, while documents to be archived or dismissed are managed through `addDocument` and `removeDocument`. The objects that respectively represent users and documents belong to the two classes `User` and `Document`. As apparent from the class diagram, there are two kinds of users: normal users, represented as objects of the base class `User`, and internal users, represented by the subclass `InternalUser`. Library documents are also classified into categories. A library can manage journals (class `Journal`), books (class `Book`), and technical reports (class `TechnicalReport`). All these classes extend the base class `Document`.

The attributes of class `User` aim at storing personal data about library users, such as their full name, address and phone number. A user code (attribute `userCode`) is used to uniquely identify each user. This could be read from a card issued to library users (e.g., reading a bar code). In addition to that, internal users are identified by an internal code (attribute `internalId` of class `InternalUser`).

Objects of class `Document` are identified by a code (attribute `documentCode`), and possess attributes to record the title, authors and ISBN code. Technical reports obey an alternative classification scheme, being identified also by their reference number (attribute `refNo`).

A `Library` holds the list of its users and documents. This is represented in the class diagram by the two associations respectively toward classes `User` and `Document` (labeled `users` and `documents`, resp.). These associations provide a stable reference to the collection of documents and the set of users currently handled.

The process of borrowing a document is objectified into the class `Loan`. A `Library` manages a set of current loans, indicated in the class diagram as an association toward class `Loan` (labeled `loans`). A `Loan` consists of a `User` (association labeled `user`) and a `Document` (association `document`). It represents the fact that a given *user* borrowed a given *document*. A `Library` can access the list of its active loans through the association `loans` and from each `Loan` object, it can obtain the `User` and `Document` involved in the loan.

The two associations, between `Loan` and `User`, and between `Loan` and `Document`, are made bidirectional by the addition of a reverse link (from `User` to `Loan` and from `Document` to `Loan` resp.). This allows getting the set of loans of a given user and the loan (if any exists) associated to a given document. The chain from users to documents, and vice versa, can thus be closed. Given a user, it is possible to access her/his loans (association `loans`), and from each loan, the related `Document` object. In the other direction, given a `Document`,

it is possible to see if it is borrowed (association `loan` leads to a non-null object), and in case a `Loan` object exists, the user who borrowed the document is accessible through the association `user` (from `Loan` to `User`).

Class `Library` establishes the relationships between users and documents, through `Loan` objects, when calls to its method `borrowDocument` are issued. On the contrary, the method `returnDocument` is responsible for dropping `Loan` objects, thus making a document no longer connected to a `Loan` object, and diminishing the number of loans a user is associated with. When a document is requested for loan by a user, the `Library` checks if it is available, by invoking the method `isAvailable` of class `Document`, and if the given user is authorized to borrow the document, by invoking the method `authorizedLoan` inside class `Document`. Since loan authorization depends also on the kind of user issuing the request (normal vs. internal user), a method `authorizedUser` is provided inside the class `User` to distinguish normal users from users with special loan privileges. The method `authorizedLoan` is overridden when the default authorization policy, implemented by the base class `Document`, needs be changed in a subclass (namely, `TechnicalReport` and `Journal`). Similarly, the default authorization rights of normal users, defined in the base class `User`, are redefined inside `InternalUser`.

Search facilities are available inside the class `Library`. Users can be searched by name (method `searchUser`), while documents can be searched by title (method `searchDocumentByTitle`), authors (method `searchDocumentByAuthors`), or ISBN code (method `searchDocumentByISBN`). Retrieved users can be associated with the documents they borrowed and retrieved documents can be associated with the users who borrowed them (if any) as explained above.

Print facilities are available inside classes `Library`, `User`, `Document`, and `Loan` (for clarity, some of them are not shown in Fig. 1.1). The method `printInfo` is a function to print general information available from the classes `User` and `Document`. The method `printAvailability` inside class `Document` emits a message stating if a given document is available or was borrowed. In the latter case, information about the user who borrowed it is also printed.

The mutual dependencies between classes `User` and `Document` (dashed lines in Fig. 1.1) are due to the invocation of methods to gather information that is displayed by some printing function. For example, the method `printInfo` of class `User` displays personal user data, followed by the list of borrowed documents. Information about such documents is obtained by traversing the two associations `loans` and `document`, leading to a `Document` object for each borrowed item. Then, calls to get data about each `Document` (e.g., method `getTitle`) are issued. Hence, the dependency from `User` to `Document`. Symmetrically, method `printAvailability` of class `Document` accesses user data (e.g., calling method `getName`), in case a `User` borrowed the given `Document`. This happens when the association `loan` is non-null. The direct invocation from `Document` to `User` is the cause of the dependency between these two classes.

Authorization to borrow documents is handled in a straightforward way inside the classes `Document` and `TechnicalReport`, which return a constant value (resp. `true` and `false`) and do not use at all the parameter `user` received upon invocation of `authorizedLoan`. On the other side, the class `Journal` returns a value that depends on the privileges of the parameter `user`. This is achieved by calling `authorizedUser` from `authorizedLoan` inside `Journal`. This direct call from `Journal` to `User` explains the dependency between these two classes in the class diagram.

Chapter 3 provides an algorithm for the extraction of the class diagram in a context similar to that of the *eLib* program, where weakly typed containers and interfaces are used in attribute and variable declarations.

## 1.4 Object Diagram

The object diagram focuses on the objects that are created inside a program. Most of the object creations for the classes in the *eLib* program are performed inside an external driver class, such as that reported in Appendix B.

The *static* object diagram represents all objects and inter-object relationships possibly created in a program. The *dynamic* object diagram shows the objects and the relationships that are created during a specific program execution.

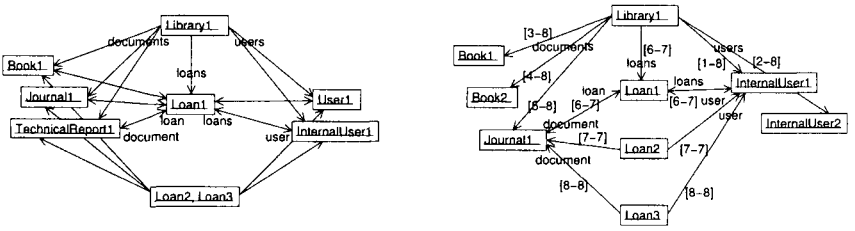


**Fig. 1.2.** Static (left) and dynamic (right) object diagram for the *eLib* program.

Fig. 1.2 depicts both kinds of object diagrams for the *eLib* program. In the static object diagram, shown on the left, each object corresponds to a distinct allocation statement in the program. Thus, for the *eLib* program under analysis (Appendixes A and B), there is one allocation point for creating objects of the classes `Library`, `Book`, `Journal`, `TechnicalReport`, `User`, `InternalUser`. No object of class `Document` is ever allocated, while objects of class `Loan` are allocated by three different statements inside the class `Library`. One such allocation (line 60) belongs to the method `borrowDocument`, and produces the object named `Loan1`, another one (line 70) is inside `returnDocument` and produces `Loan2`, while the third one (line 78), inside `isHolding`, produces `Loan3`.

As apparent from the diagram in Fig. 1.2 (left), the object allocated inside `borrowDocument` (`Loan1`) is contained inside the list of loans possessed by the object `Library1`, which represents the whole library. `Loan1` references the document and the user participating in the loan. These are objects of type `Book`, `Journal`, `TechnicalReport` and `User`, `InternalUser` respectively, as depicted in the static object diagram. In turn, they have a reference to the loan object (bidirectional link in Fig. 1.2). On the contrary, the objects `Loan2` and `Loan3` are not accessible from the list of loans held by `Library1`. They are temporary objects created to manage the deletion of a loan (method `returnDocument`, line 70) and to check the existence of a loan between a given user and a given document (method `isHolding`, line 78). However, none of them is in turn referenced by the associated user/document (unidirectional link in Fig. 1.2).

The dynamic object diagram on the right of Fig. 1.2 was obtained by executing the *eLib* program under the following scenario:

| Time | Operation |
|---|---|
| 1 | An internal user is registered into the library. |
| 2 | Another internal user is registered. |
| 3 | A book is archived into the library |
| 4 | Another book is archived. |
| 5 | A journal is archived into the library. |
| 6 | The journal archived at time 5 is borrowed by the first registered user. |
| 7 | The journal borrowed at time 6 is returned to the library and the loan is closed. |
| 8 | The librarian verifies that the loan was actually closed. |

The time intervals indicating the life span of the inter-object relationships are in square brackets. The objects `InternalUser1`, `InternalUser2` represent the two users created at times 1 and 2, while `Book1`, `Book2`, `Journal1` are the objects created when two books and a journal are archived into the library, at times 3, 4, 5 respectively. When a loan is opened between `InternalUser1` and `Journal1` at time 6, the object `Loan1` is created, referencing, and referenced by, the user and document involved in the loan. At time 7 the loan is closed. Correspondingly, the life interval of all associations linked to `Loan1` is [6-7], including the association from the object `Library1`, representing the presence of `Loan1` in the list of currently active loans (attribute `loans` of the object `Library1`). Loan deletion is achieved by looking for a `Loan` object (indicated as `Loan2` in the object diagram) in the list of the active loans (`Library1.loans`). `Loan2` references the document (`Journal1`) and the user (`InternalUser1`) that are participating in the loan to be removed. Being a temporary object, `Loan2` disappears after the loan deletion operation is finished, together with its associations (life span [7-7]). The object `Loan3` has a

similar purpose. It is temporarily created to verify if `Library1.loans` contains a `Loan` which references the same user and document (resp., `InternalUser1` and `journal1`) as `Loan3`. After the check is completed, `Loan3` and its associations are dismissed (life span [8-8]).

Static and dynamic object diagrams provide complementary information, extremely useful to understanding the relationships among the objects that are actually allocated in a program. The existence of three different roles played by the objects of class `Loan` is not visible in the class diagram. It becomes clear once the object diagram for the *eLib* application is built. Moreover, the analysis of the dynamically allocated objects during the execution of a specific scenario allows understanding the way relationships are created and destroyed at run time. Temporary objects and relationships, used only in the scope of a given operation, can be distinguished from the stable relationships that characterize the management of users, documents and loans performed by the library. Moreover, the dynamics of the inter-object relationships that take place when a document is borrowed or returned also become explicit. Overall, the structure of the objects instantiated by the *eLib* program and of their mutual relationships, which is somewhat implicit in the class diagram, becomes clear in the object diagrams recovered from the code and from the program's execution.

Static and dynamic object diagram extraction is thoroughly discussed in Chapter 4.

## 1.5 Interaction Diagrams

The exchange of messages among the objects created by a program can be displayed either by ordering them temporally (sequence diagrams) or by showing them as labels of the inter-object relationships (collaboration diagrams). These are the two forms of the interaction diagrams. Each message (method call) is prefixed by a Dewey number (sequence of dot-separated decimal numbers), which indicates the flow of time and the level of nesting. Thus, a method call numbered 3.2 will be the second call nested inside another call, numbered 3.

Fig. 1.3 clarifies the interactions among objects that occur when a document is borrowed by a library user. The first three operations shown in the collaboration diagram in Fig. 1.3 (numbered 1, 2, 3) are related to the rules for document loaning implemented in the *eLib* program. In fact, the first operation (call to `numberOfLoans`) is issued from the `Library` object to the user who intends to borrow a document. The result of this operation is the number of loans currently held by the given user. The borrowing operation can proceed only if this number is below a predefined threshold (constant `MAX_NUMBER_OF_LOANS` in class `Library`).